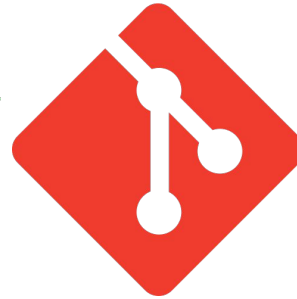


LET'S *GIT* STARTED!

INTRODUCTION TO GIT AND GITHUB



LET'S PLAN A PARTY TOGETHER!

Git

- Basic principles of Git
- The structure of the Git repository

Hands-on

add, commit, status, log, branch, merge

GitHub

Introduction GitHub

Synchronizing Git and GitHub & basic collaborative work

Hands-on

- Push your local repository to GitHub
- Fetch and pull from a remote repository
- Fork and/or clone a project



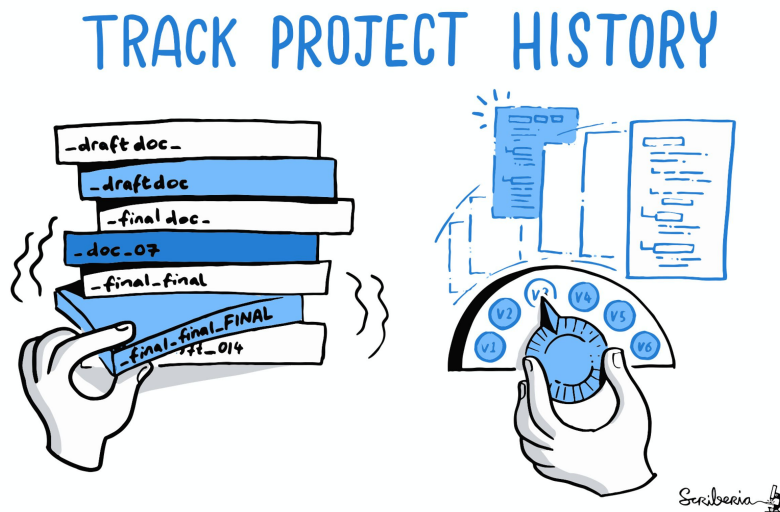
INTRODUCTION TO VERSION CONTROL

Why use Git?

You're working on a research project containing analysis code

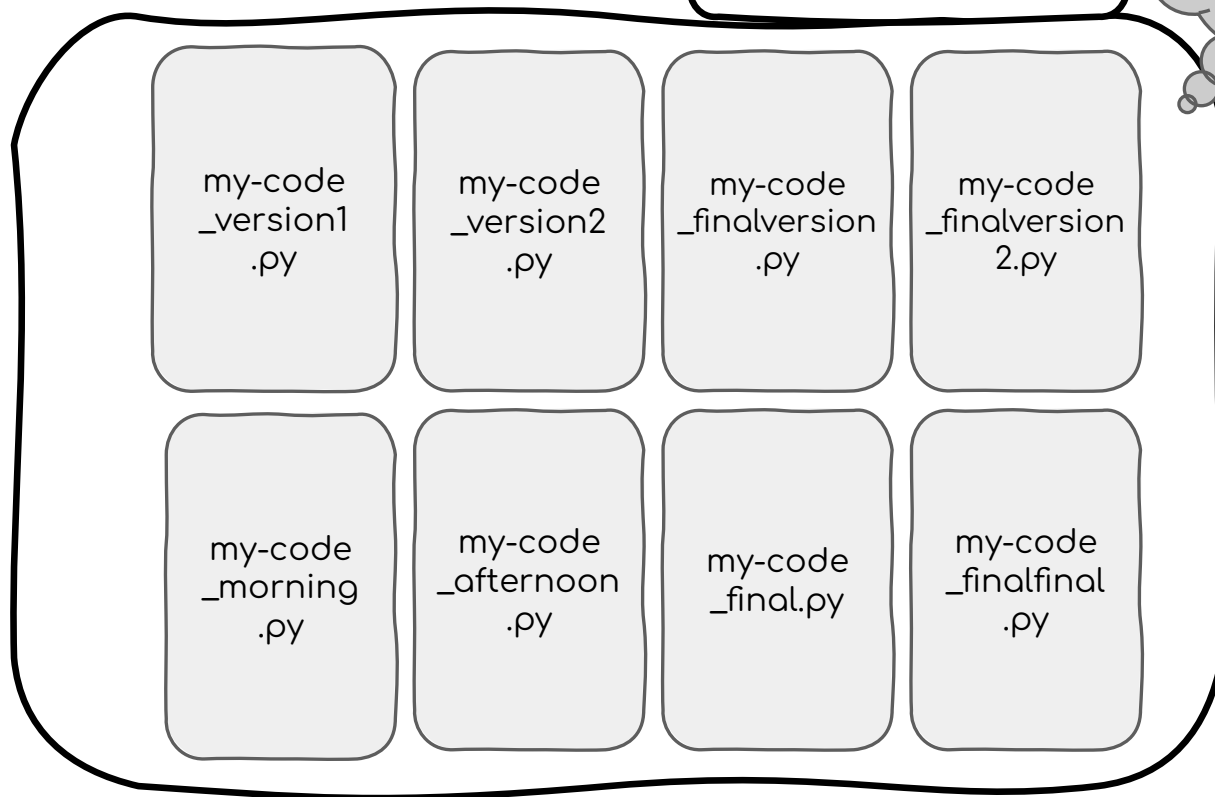
- you want to test different versions of one code file before settling on the best approach
- you want to test different versions of a combination of code files

How do you keep track of the modifications you made?



Example

without version control:



What version do I need? Which version is my co-worker currently working on?

Example

without version control:

my-project

my-code.py

How did the code
look like
yesterday?

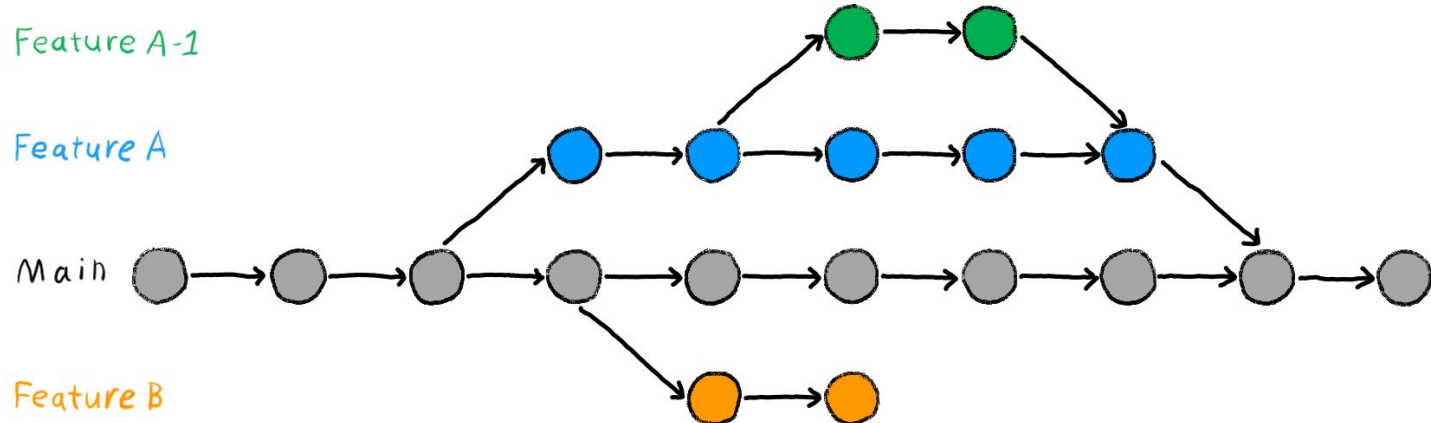
Benefits of version control

- Traceability

```
d608377 (HEAD -> sensitivity-analysis, site-weighted-ecdf, private-repo) resolve merge conflict
a966874 sync
aad3b0a (private/private-repo) add final models and embedding visuals
7ecc3e8 after fine-tuning and first results
334df48 more test runs
baacdb9 first harmonization test result
938b05a add first test items
bb30bd4 add harmonized inventory
39febbb after grid-search and hyperparameter tuning
33c48e1 mod .gitignore
33eb6fa fix code and add output
7b27fe3 add hyperparameter code
676e7cb add fine-tuning results
87919e6 modify gitignore
8dbeff8 update semantic learning script
9bc6539 re-run all analysis with corrected data input and corrected expert assignments
783995a add fine-tuning output to gitignore because too big for github
31972a9 correct data_items-and-ratings_clean.csv and run expert analysis again
1b5736e (main) add input and output folders for private repo
07895d6 (assignments-corrected) add similarity analysis correction code and fine-tuning code
f0e91a2 (cleaning, apathy-collapsed) delete unnecessary files and code
3c4e659 move old code
0b18c6b save code with new confidence evidence scores
57d6bec (origin/main) reset local repo
47eef7e Merge branch 'main' of github.com:ENIGMA-PD/psych-in-harmony
```

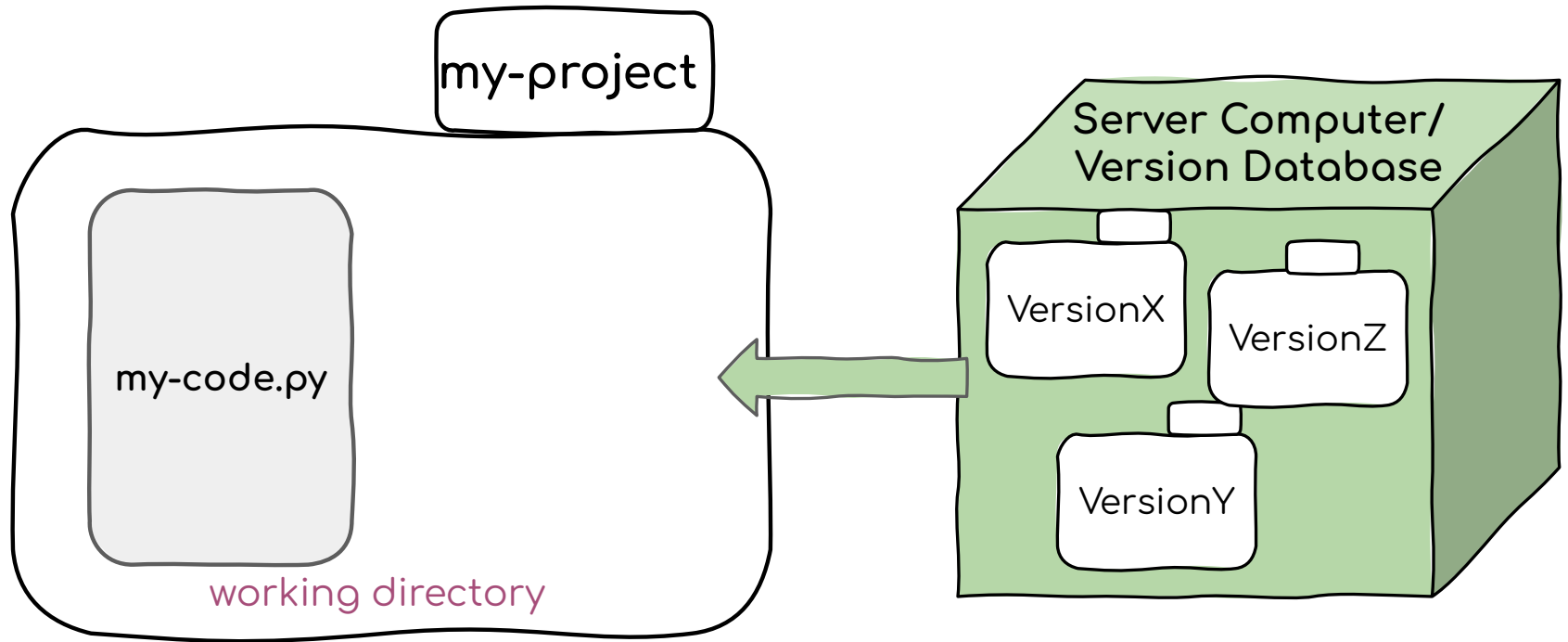
Benefits of version control

- Traceability
- multiple streams of work (or version sets) independent from each other while also being able to merge the work back together

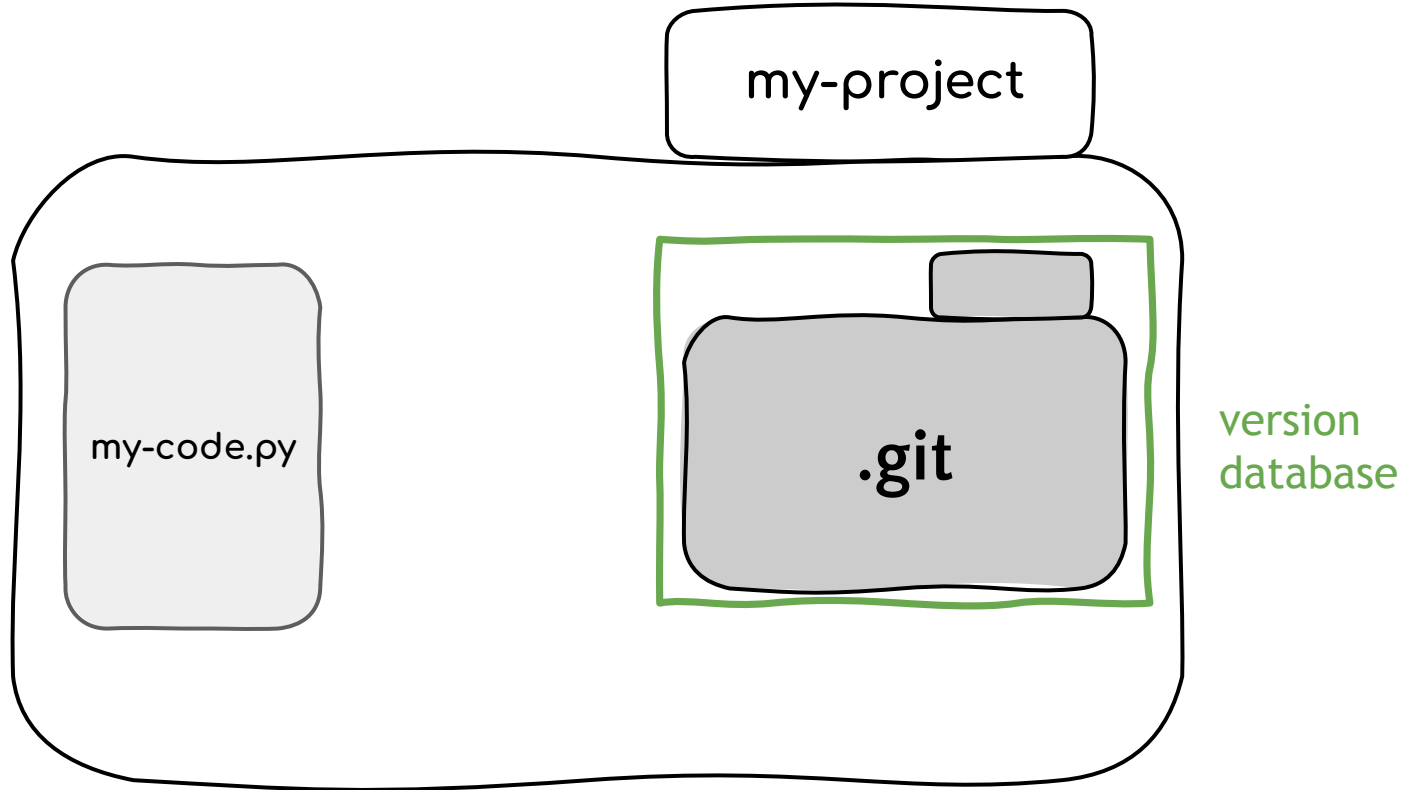


Example

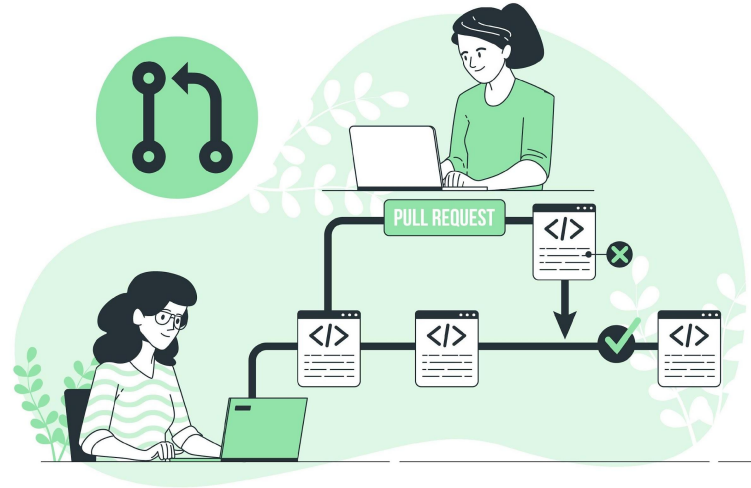
with version control:



The Git repository

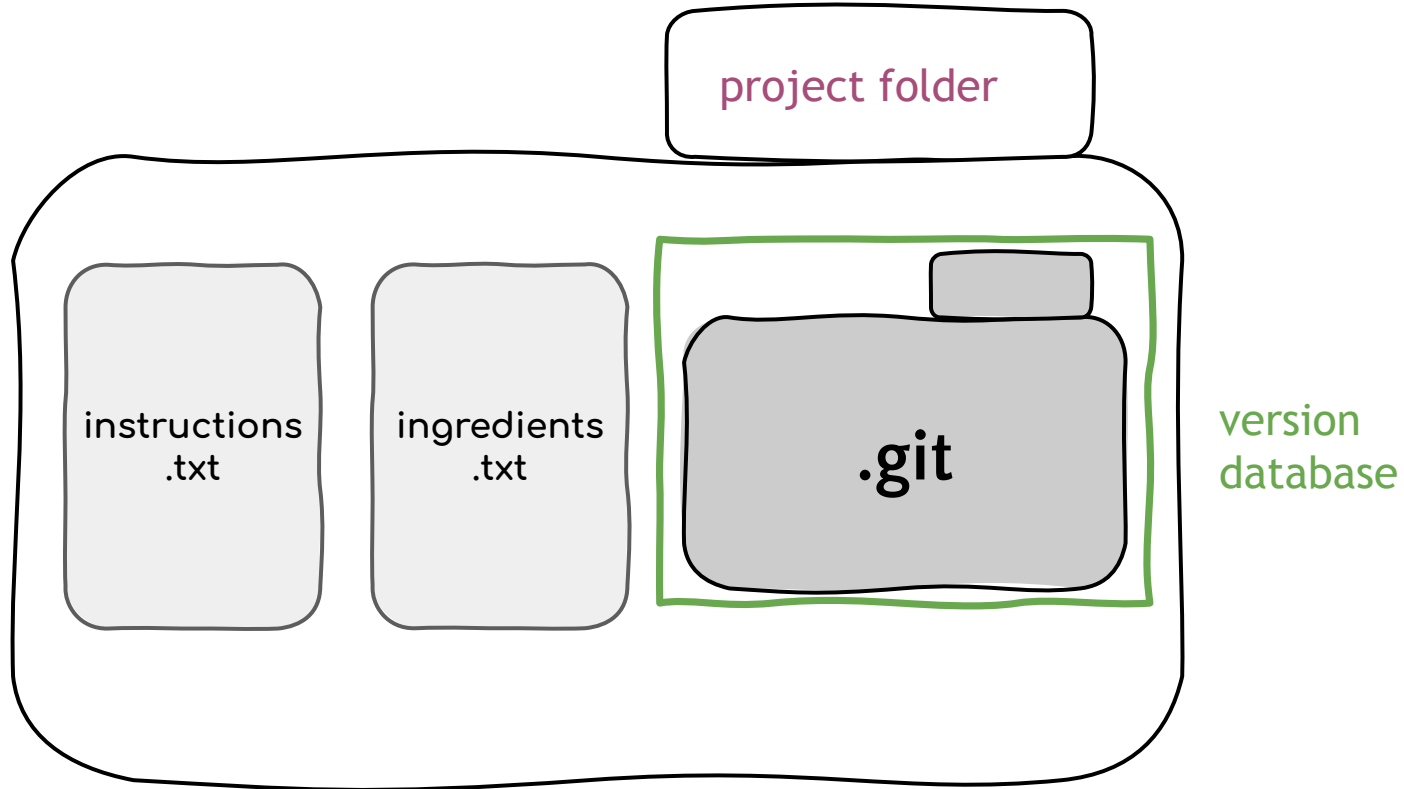


Benefits of version control

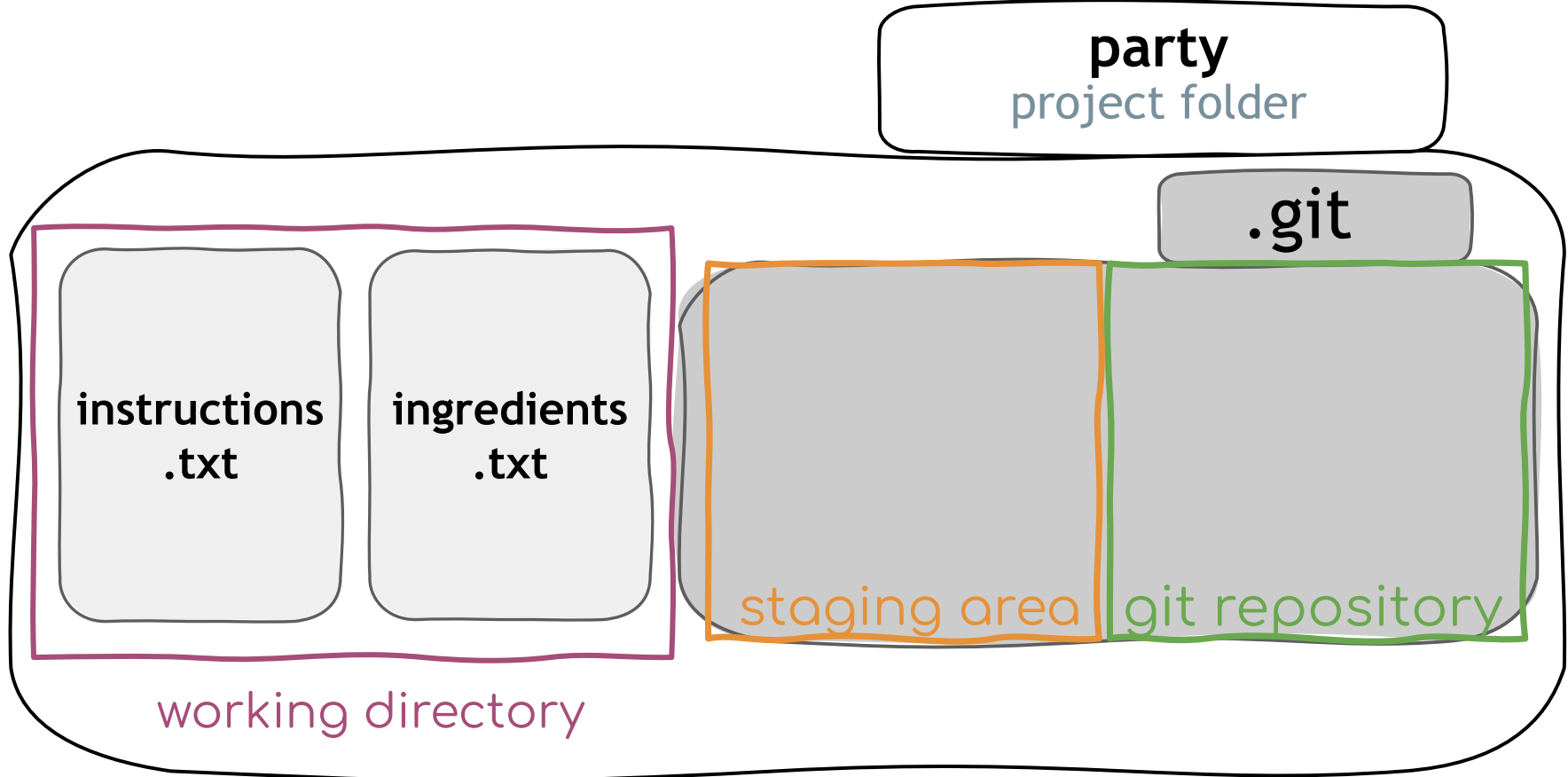


HOW DOES GIT WORK?

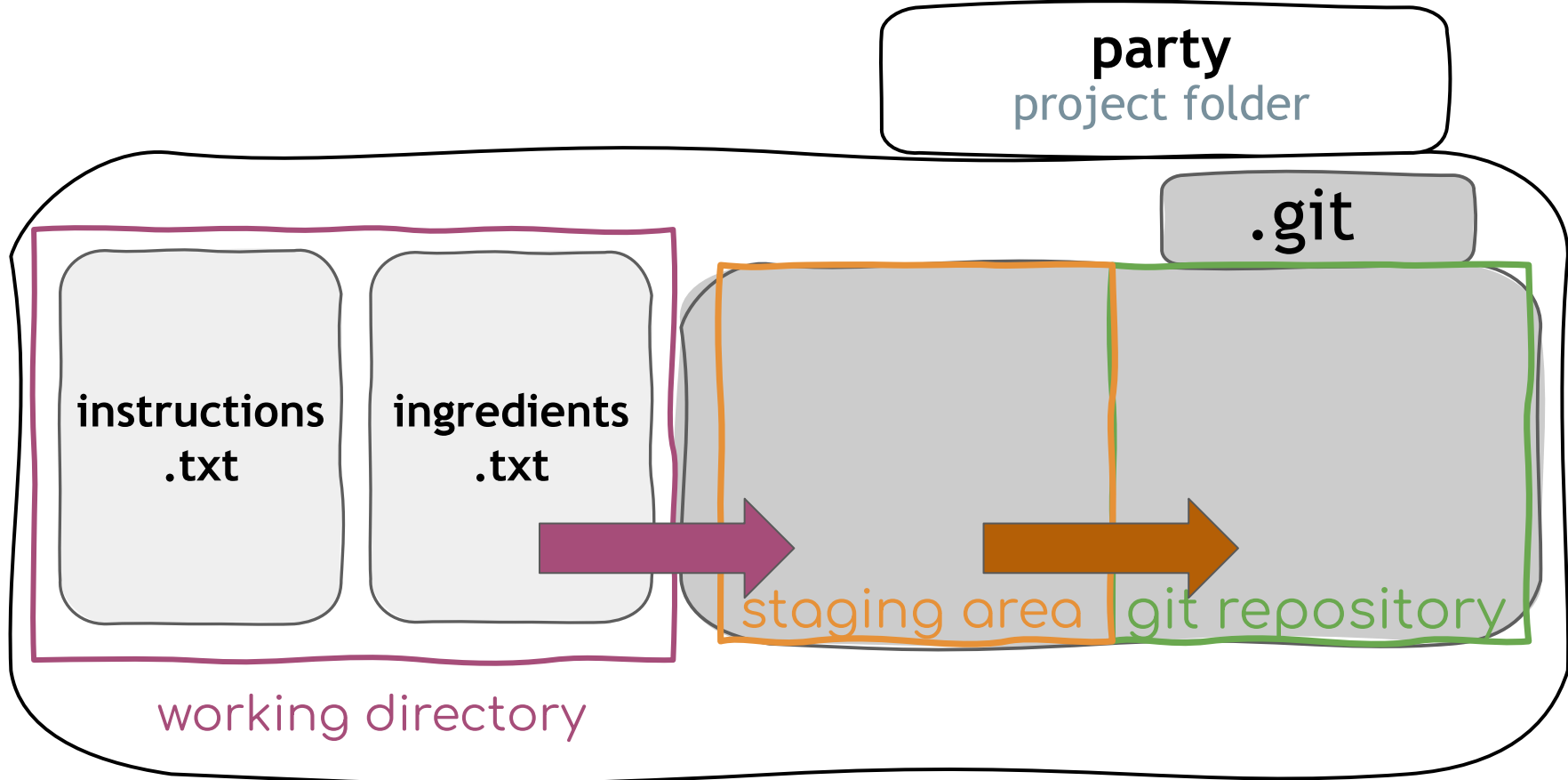
The structure of the version database



The structure of the version database



The structure of the version database



The staging area



- Changes on a file need to be first added to the staging area (changes in files can be added individually)
- “staging a file for a commit” means that this file will be included in your next commit
- changes that are not staged will not be included in your next commit → those changes will only stay in your working directory but not saved in the git repository

Why first add before commit?

- when you make a commit, git asks you to give a short description of what this commit is about → this **commit message** is bound to your commit/hash
 - e.g., “Wrote code for preprocessing steps 1&2 according to some paper”
- before making a commit, you should think about the usefulness and logic of this commit
 - e.g., do I really need an extra commit/hash for the change from “brain” to “brains”?
 - can I describe these multiple changes together?
 - customize what goes into a commit!

COMMITTS



- making a commit
 - = taking a snapshot of the current stage of your project and saving it in the **git repository**
 - = commits are stand alone versions of the project
- for every commit, git creates a **hash** which looks like this:
7c35a3ce607a14953f070f0f83b5d74c2296ef93
- all hashes can be found in the **commit history** and can be checked-out whenever

COMMITS



```
d608377 (HEAD -> sensitivity-analysis, site-weighted-ecdf, private-repo) resolve merge conflict
a966874 sync
aad3b0a (private/private-repo) add final models and embedding visuals
7ecc3e8 after fine-tuning and first results
334df48 more test runs
baacdb9 first harmonization test result
938b05a add first test items
bb30bd4 add harmonized inventory
39febbb after grid-search and hyperparameter tuning
33c48e1 mod .gitignore
33eb6fa fix code and add output
7b27fe3 add hyperparameter code
676e7cb add fine-tuning results
87919e6 modify gitignore
8dbeff8 update semantic learning script
9bc6539 re-run all analysis with corrected data input and corrected expert assignments
783995a add fine-tuning output to gitignore because too big for github
31972a9 correct data_items-and-ratings_clean.csv and run expert analysis again
1b5736e (main) add input and output folders for private repo
07895d6 (assignments-corrected) add similarity analysis correction code and fine-tuning code
f0e91a2 (cleaning, apathy-collapsed) delete unnecessary files and code
3c4e659 move old code
0b18c6b save code with new confidence evidence scores
57d6bec (origin/main) reset local repo
47eef7e Merge branch 'main' of github.com:ENIGMA-PD/psych-in-harmony
```

BASIC GIT WORKFLOW

commit
message

*„Me and my best
friend July 2015 in
Italy“*



git add



git commit



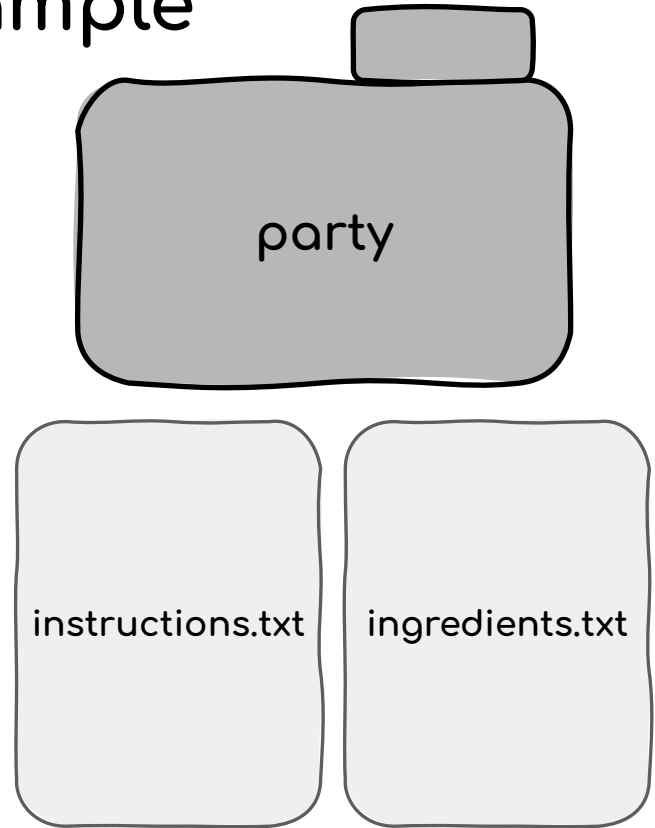
git repository

Git in practice

Our today's working example

Our project starting point:

- We have a local folder (party 🎉)
- In this folder we create two files
- Over time, we modify the contents of the files and folder



Hands-on

- `git init`
- `git add`
- `git commit`
- `git status`

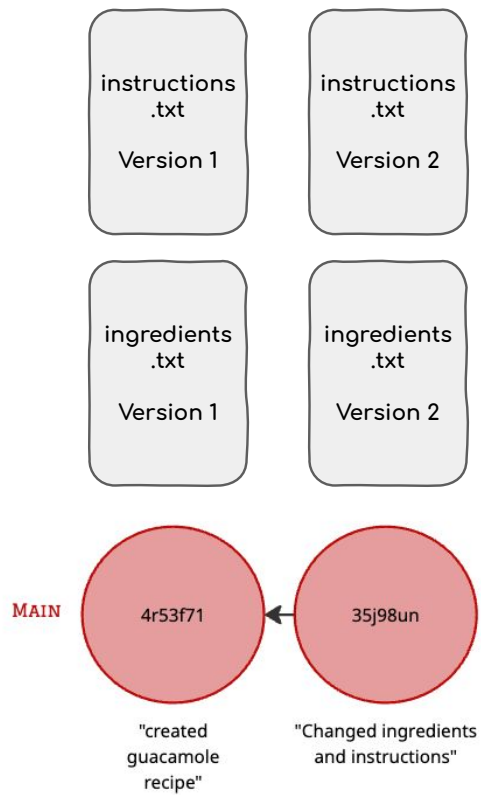
instructions
.txt
Version 1

ingredients
.txt
Version 1

MAIN

4r53f71

"created
guacamole
recipe"



Hands-on

- `git diff`
- `git log`



Branching

- The git repository can store multiple sets of versions of our project
- These sets can live parallel to each other and/or be merged together
 - One set is called a branch
- A new branch can be created from any commit
- Branches can be used as movable pointers to work on different development streams (jump between sets)

Hands-on

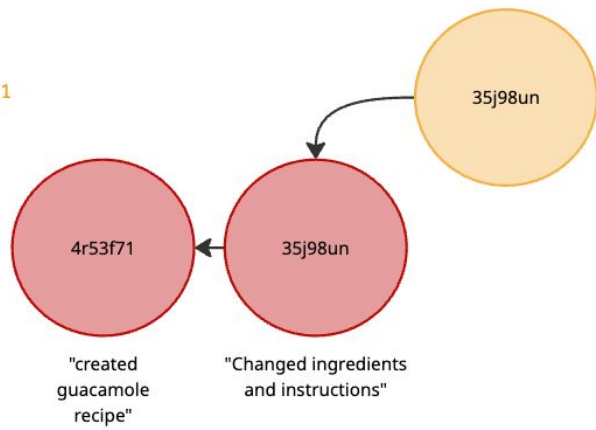
- `git branch` or `git checkout -b`

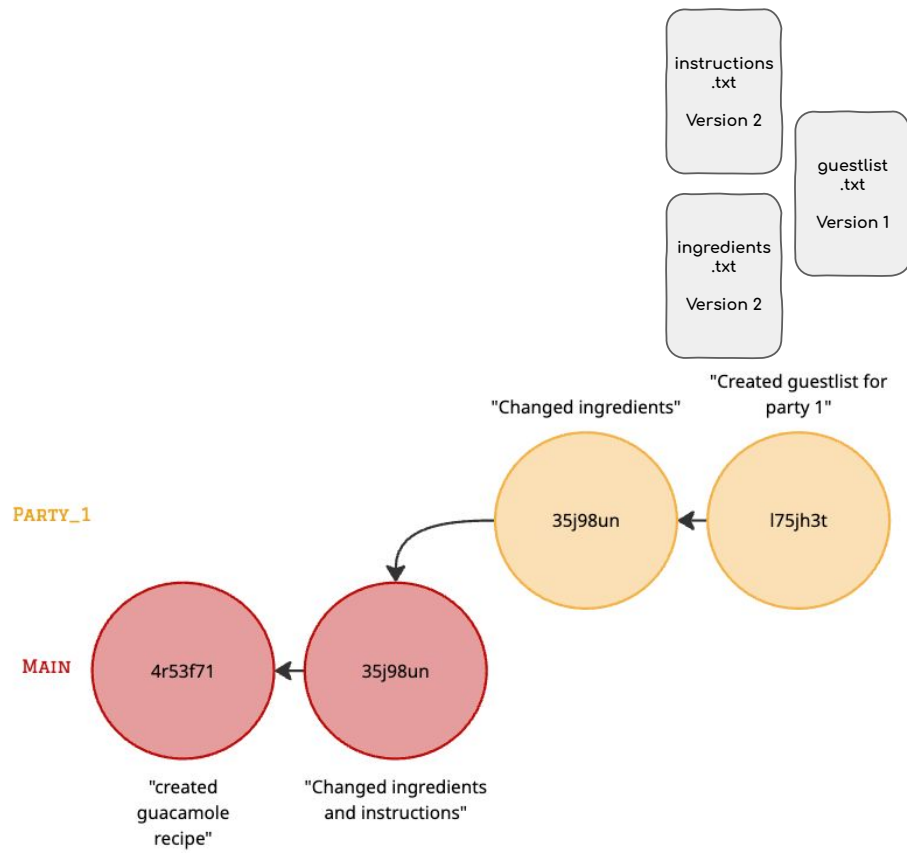


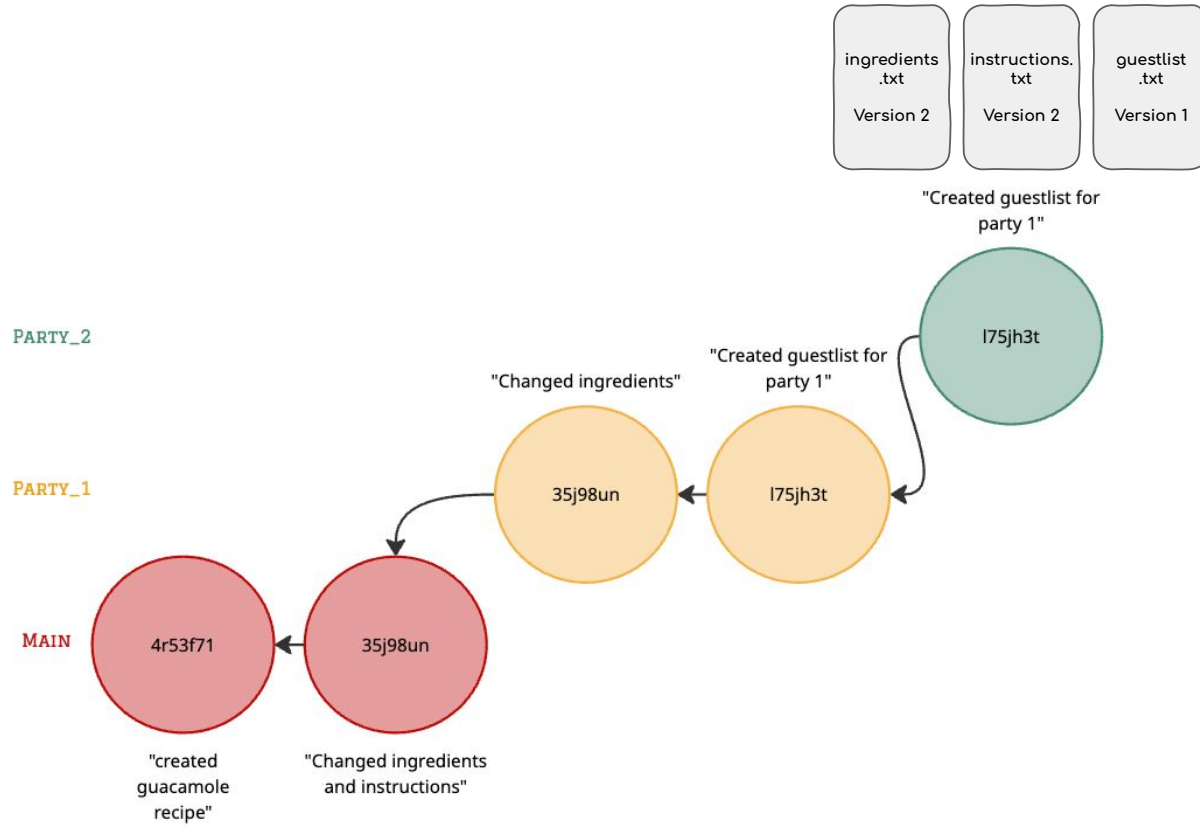
"Changed ingredients"

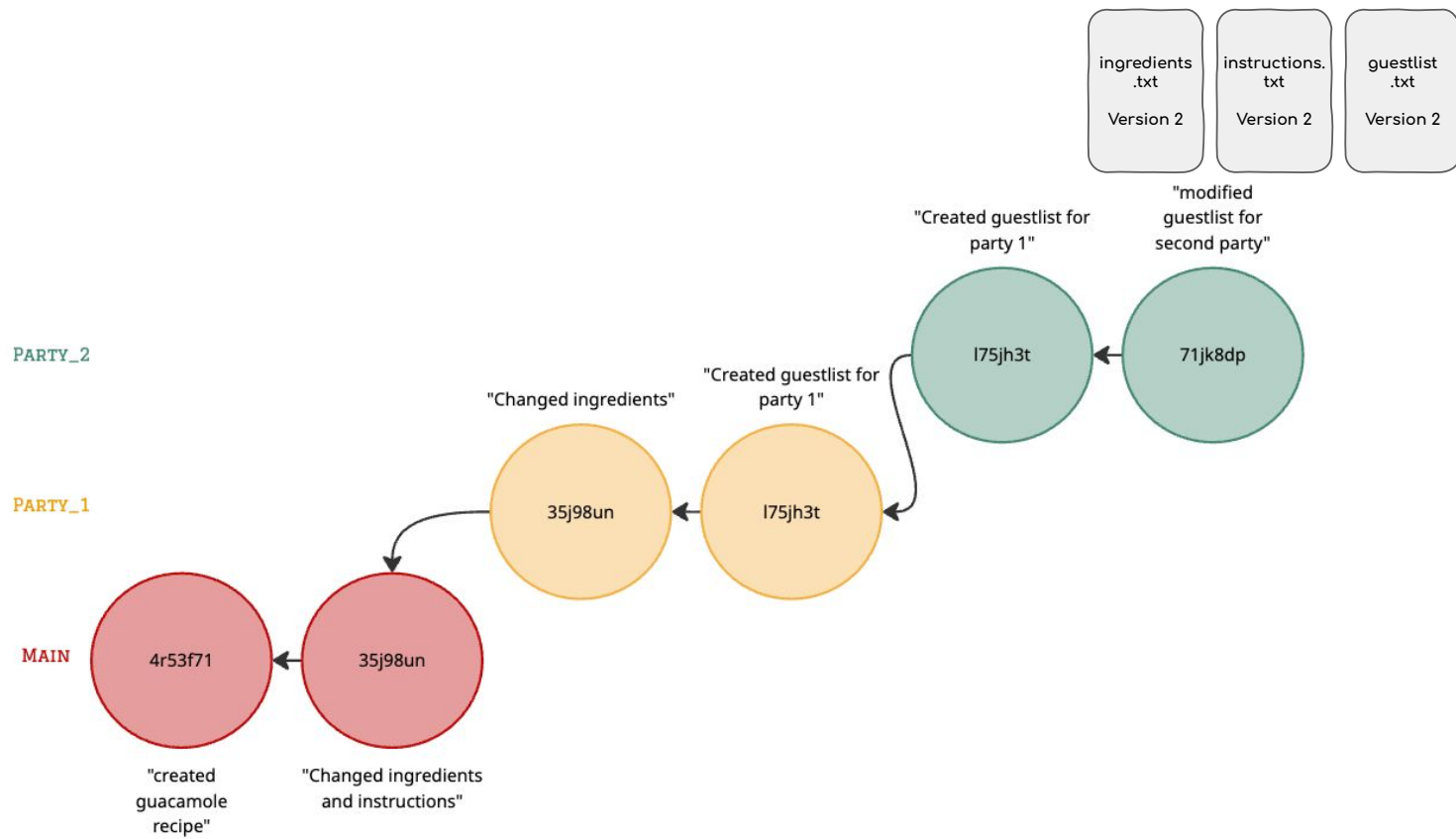
PARTY_1

MAIN



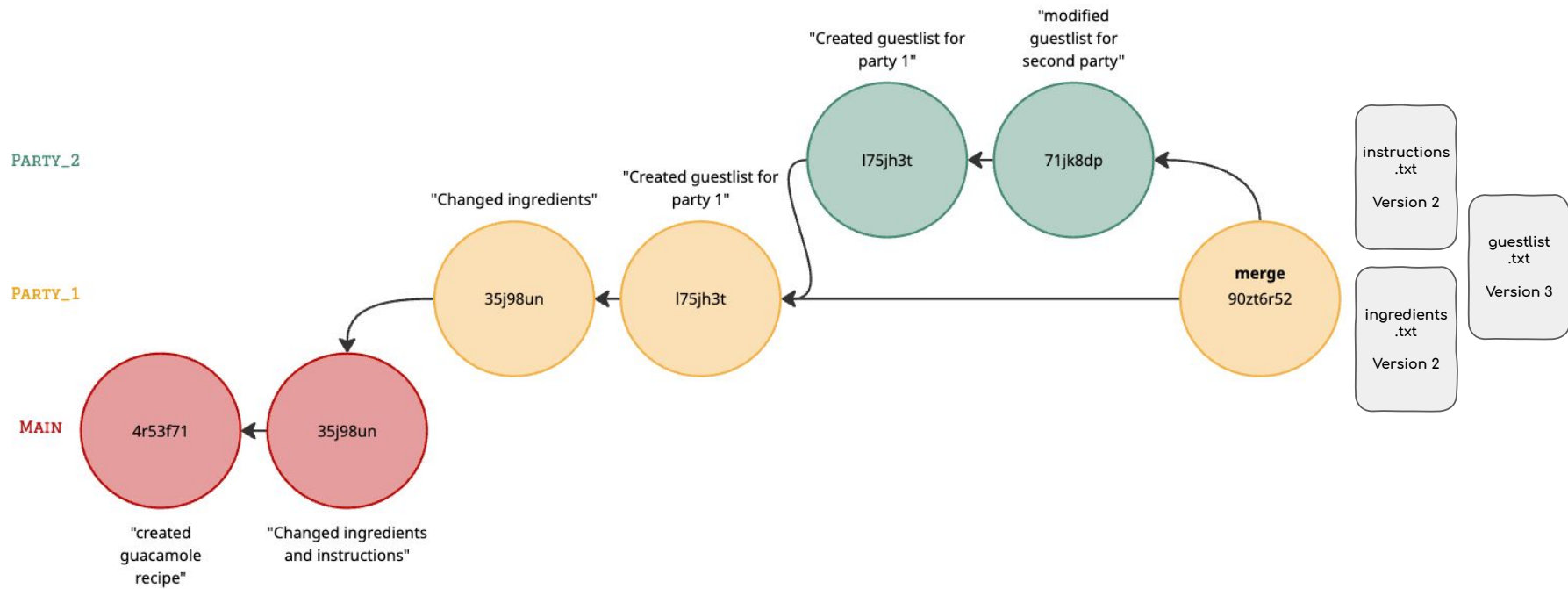




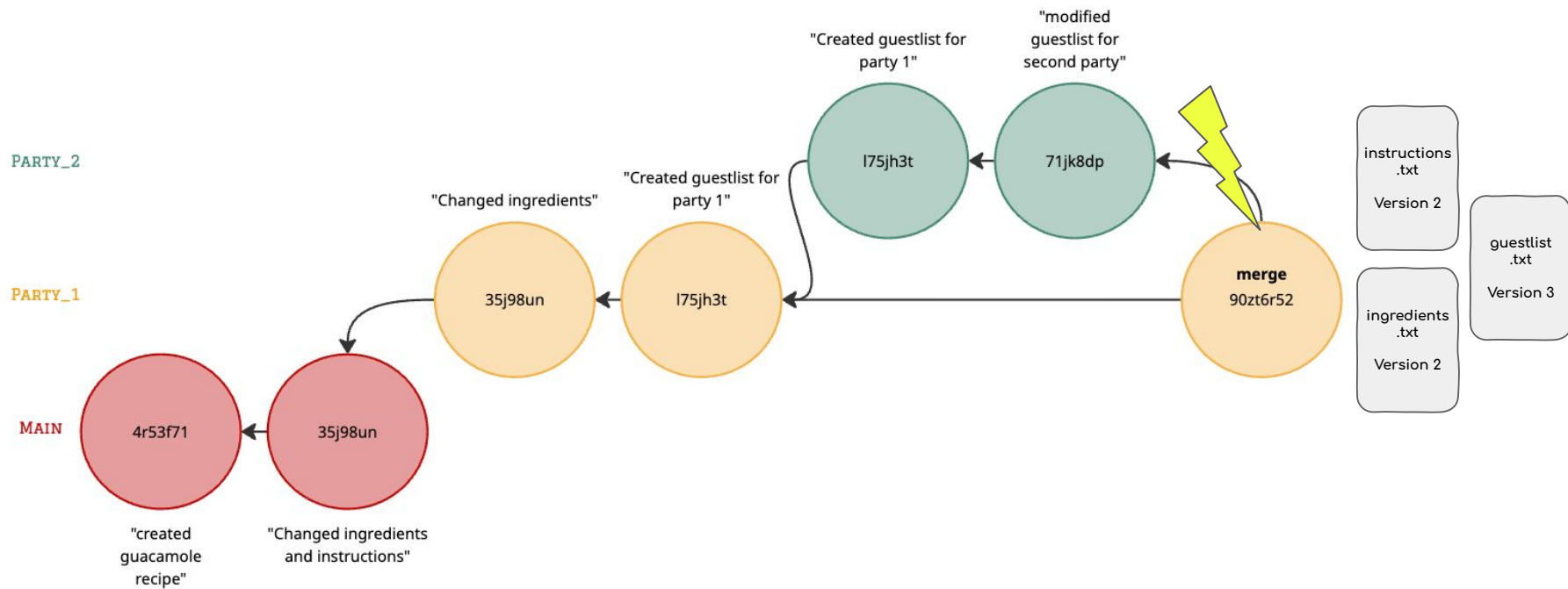


Hands-on

- `git checkout`
- `git merge`

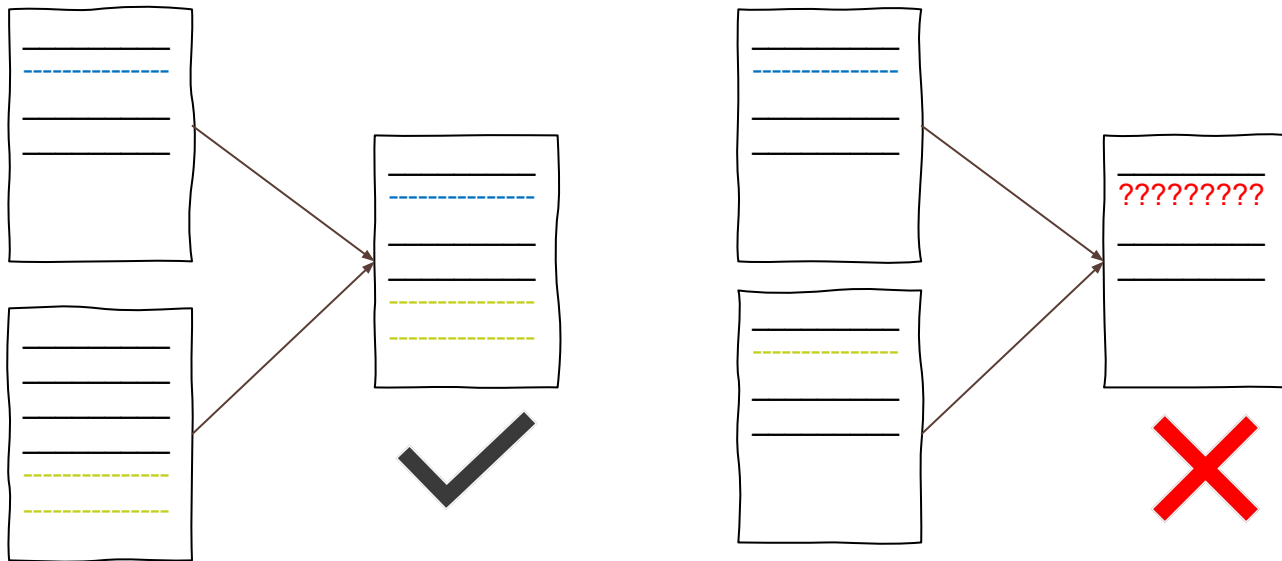


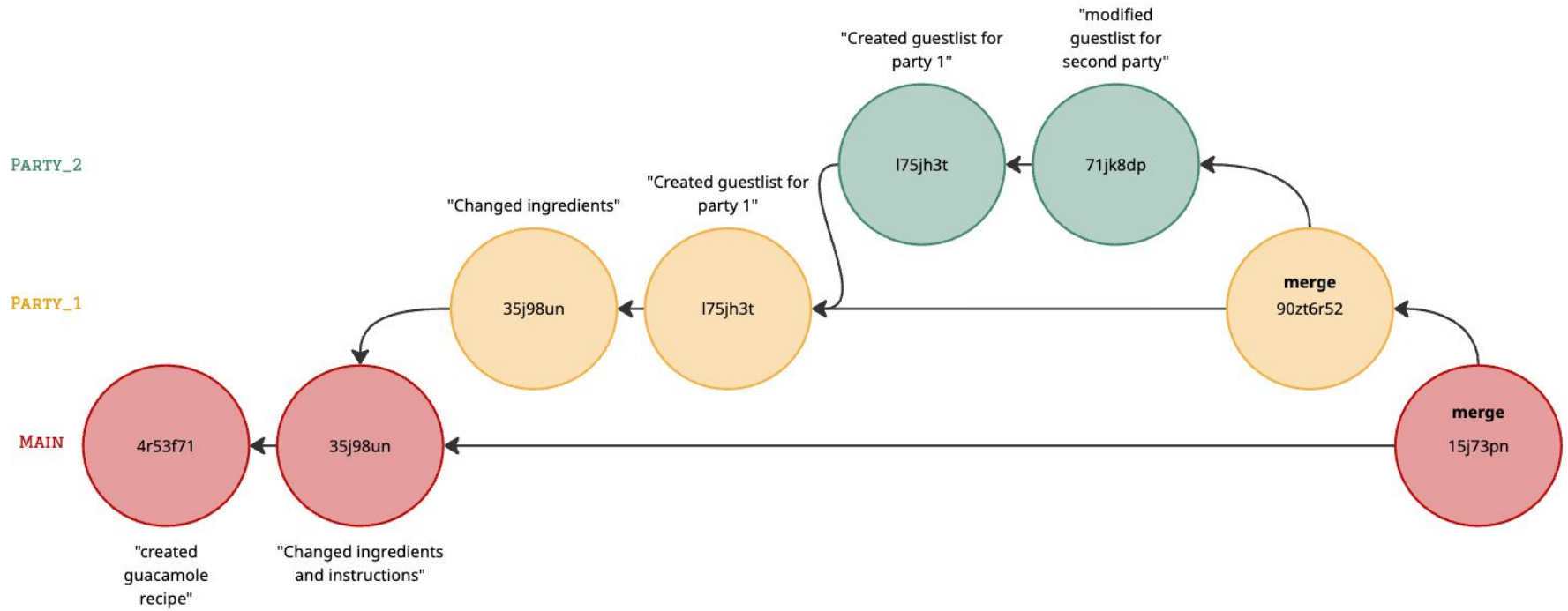
Merge conflict!



Why didn't our merge request work?

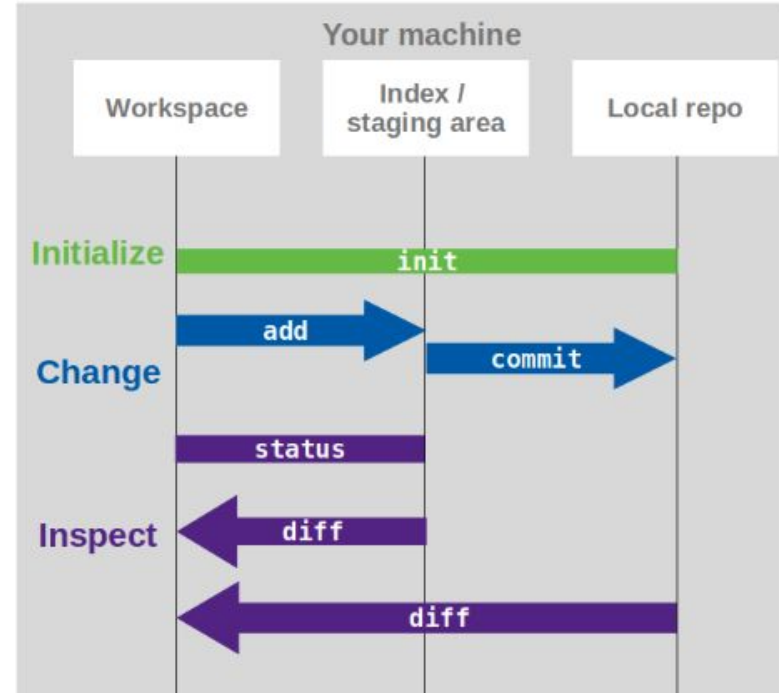
Git merges two branches on a file-by-file and line-by-line basis





What did we do so far?

- Initialized a git repository
- Made changes on our project
- Staged those changes for commit
- Saved those changes in our local git repository
- Inspected differences between versions of our project
- looked at the commit history
- merged two branches



Git vocabulary

- `git init`: to give git initial control over your files
- `git add`: adds a change from the working directory into the staging area
- `git commit`: saves a snapshot of the current version of your project in the repository
- `git status`: check the status of the files
- `git diff`: let's you inspect differences between versions
- `git log`: to view the commit history
- `git branch` or `git checkout -b`: create a new branch
- `git checkout`: check out (or “replace”) file in working directory with another staged or committed version **AND move between branches**
- `git merge`: to merge two separate sets of version (branches) into one

WORKING COLLABORATIVELY

Git vs. GitHub

Git is independent of GitHub!



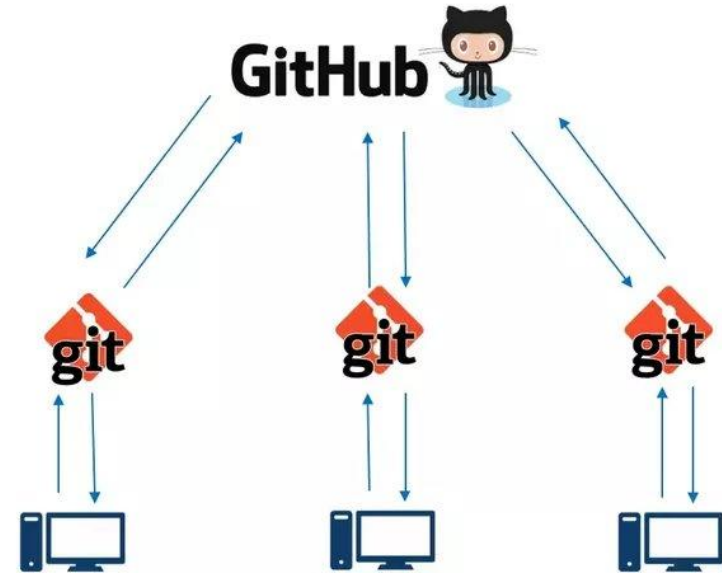
- git is a software on your computer
- decentralized
- local version control



- GitHub is a collaboration platform based on the git software
- centralized
- distributed version control

GitHub

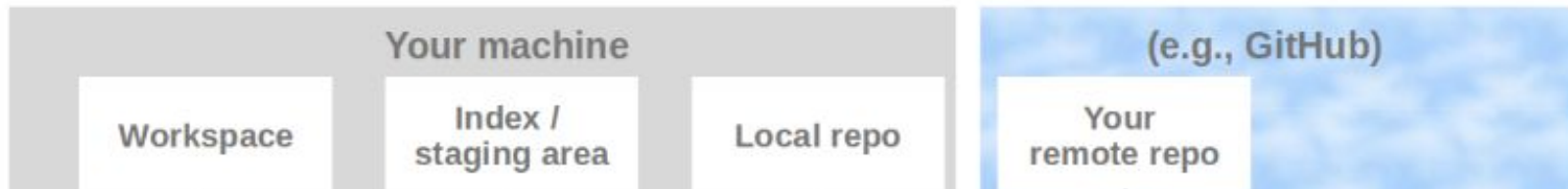
- GitHub is a collaborative (coding) platform maintained by Microsoft
- It provides you with remote repositories which you can easily synchronize with your local work
- For keeping your local Git repository and the GitHub repository synchronized, you have actively initiate the synchronization, Git does not do this automatically!
- Through the remote repository on GitHub, you can give your collaborators access to your work and updates in real-time



Local vs. remote repository

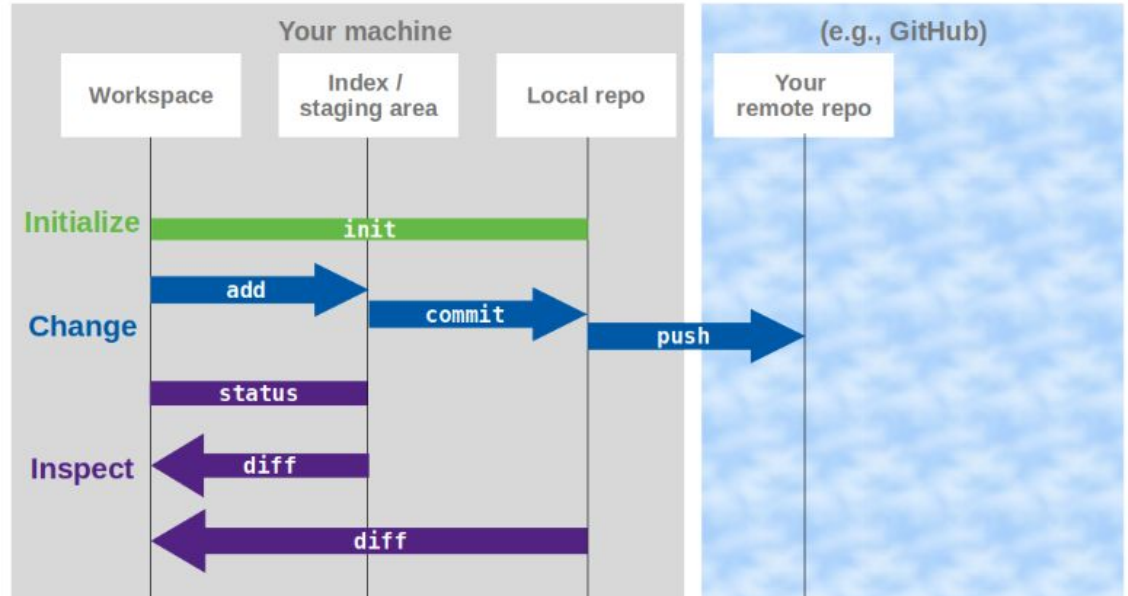
The local and the remote repository are two different things

- GitHub = your git repository online
- GitHub has its own remote branches = siblings for your local branches
- main vs. remotes/origin/main



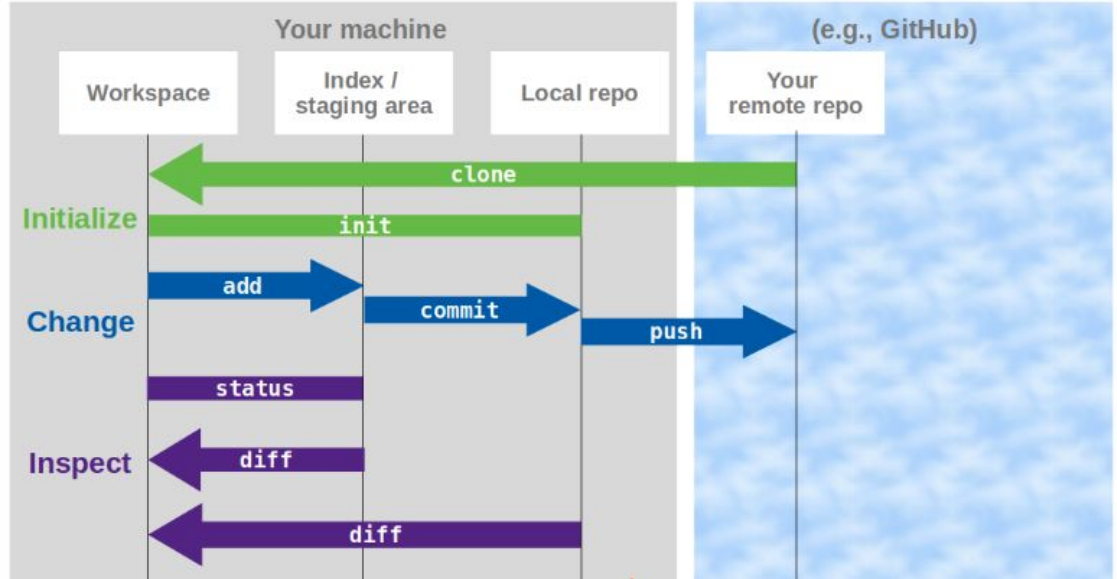
git push

synchronize your local repository with the remote repository by pushing changes made in the local repository into the remote repository



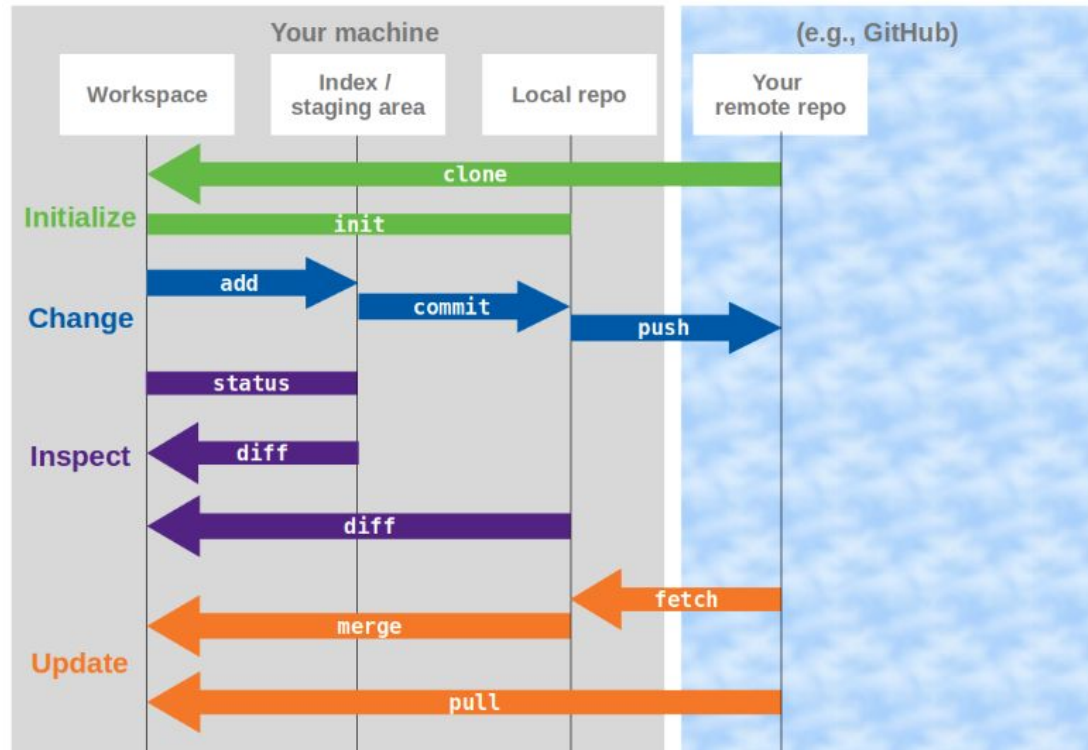
git clone

this gives you an exact copy of a remote repository on your local computer

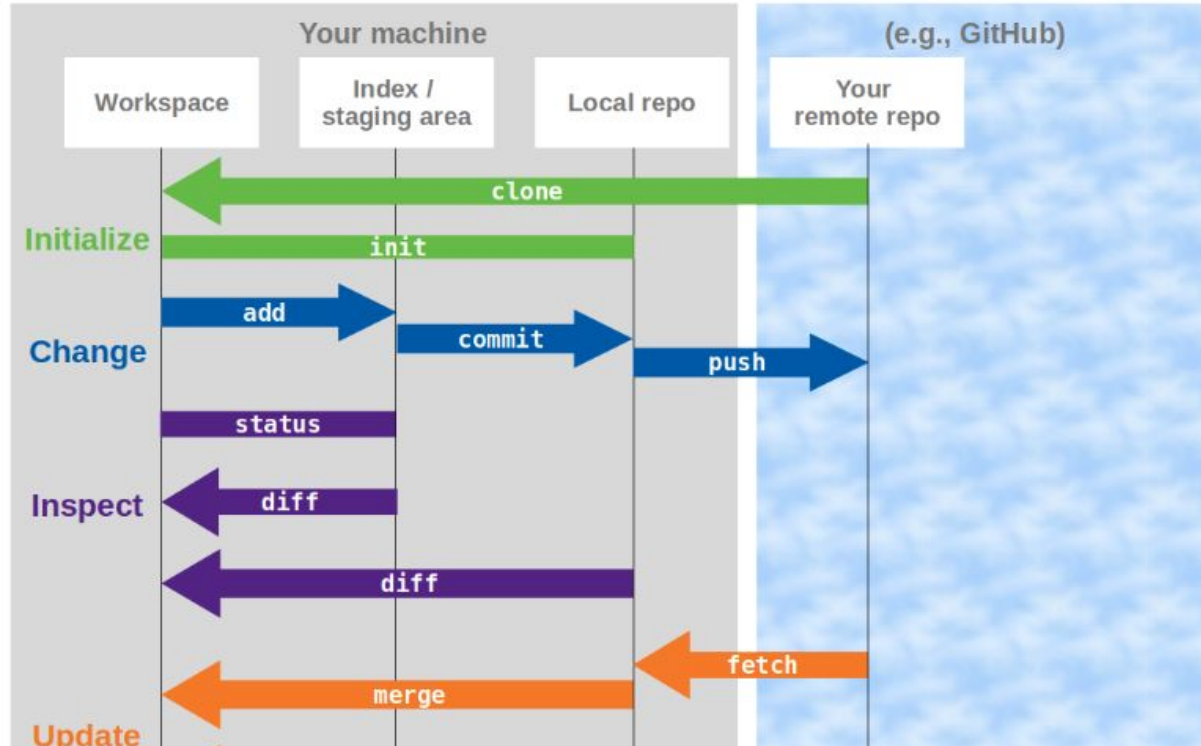


git pull

synchronize your local repository with the remote repository by pulling changes from the remote repository into the local repository

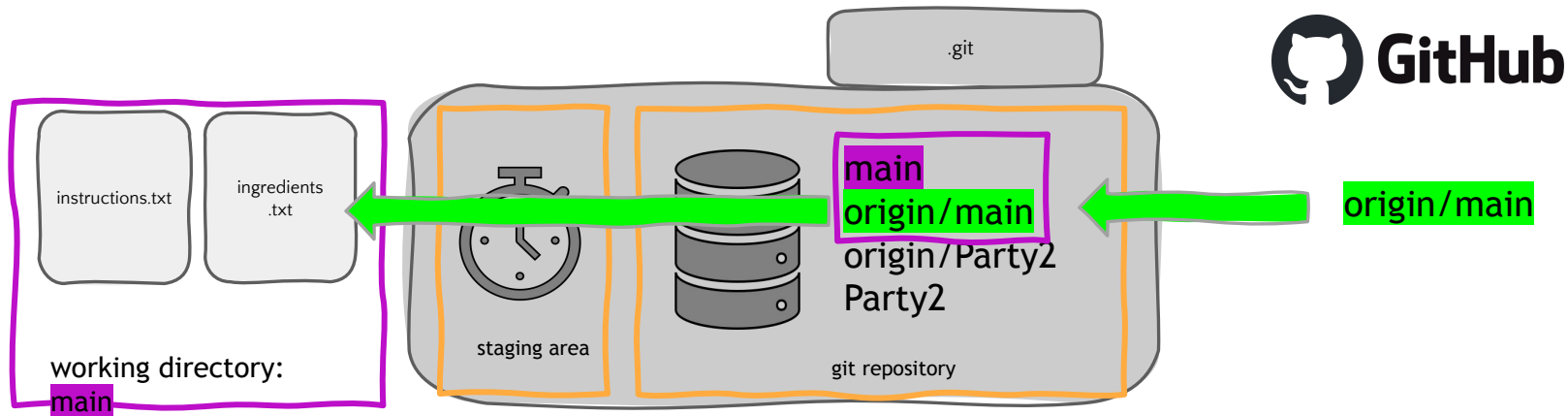


git fetch



git fetch explained

- It means that git is updating the **remote branch** in your local git repo called **"origin/main"** where it contains the latest remote version of the project
- your corresponding local branch main in your working directory might contain a different version of the project than the remote branch origin/main



→ you can do a git diff and a git merge!

difference to pull: pull pulls remote updates directly into your working directory

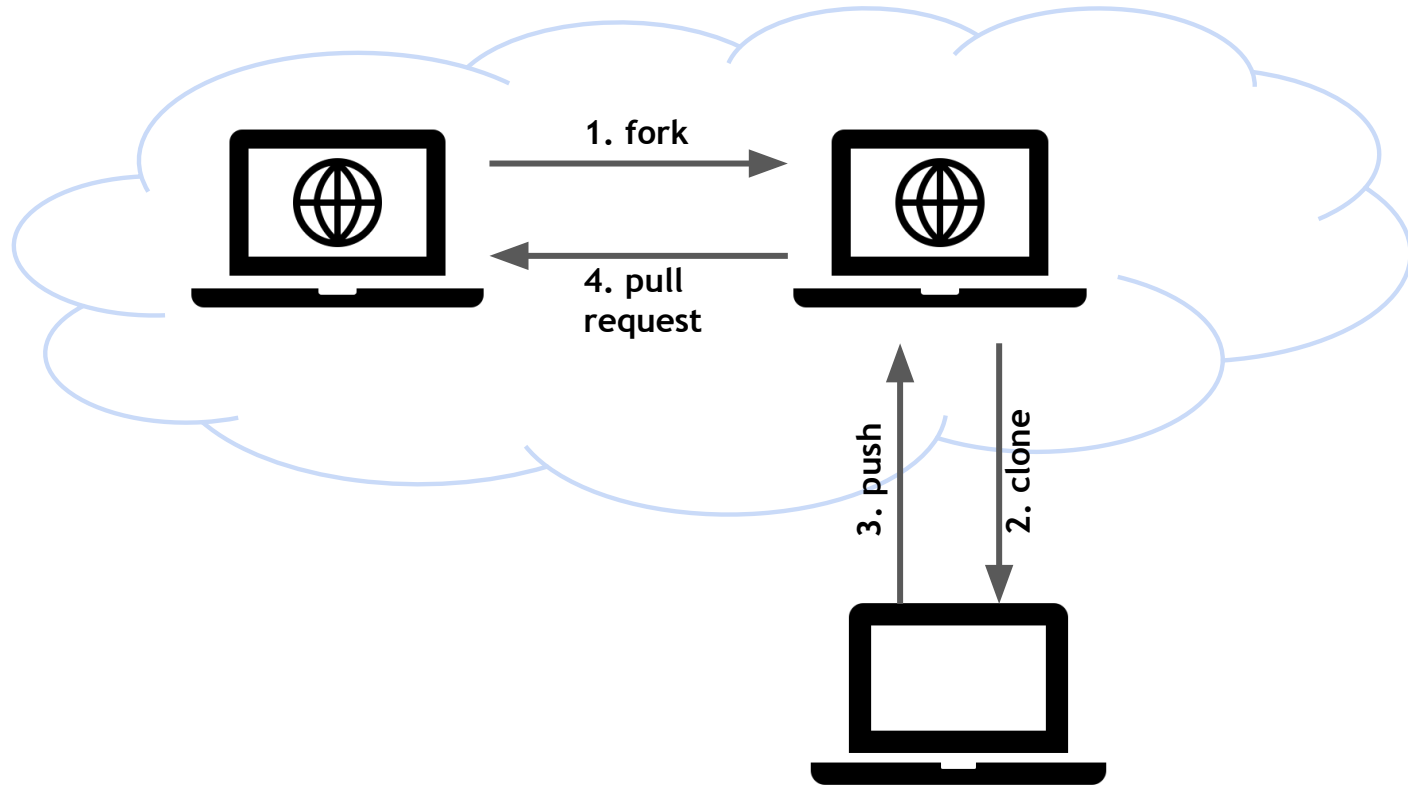
Exercise 1 - Find a partner!

Choose one of your GitHub repos to be the one you will both work on during the rest of this session
→ go to the "Settings" of this repo and add your partner with "Write" access

From now on the owner of the repository is called **owner** and the collaborator is called **collaborator**

- **collaborator** clones the repository:
 - navigate with the terminal to your desktop
 - go to the owners github repo by via search bar
 - click on the code button on the upper right
 - copy the "clone via ssh"
 - type in your terminal `git clone repo-link-you-just-copied`
 - cd in this new folder
- **collaborator** opens a file, makes a change, **adds** and **commits** it
- **collaborator** **pushes** the change to the repo by doing `git push repo-link-you-just-copied`
- **owner** refreshes GitHub repo to see if the change was updated
- **owner** needs to incorporate the changes from the remote repo into the local repo: `git pull repo-ssh-link`
- **owner** opens a file, makes a change, **adds** and **commits** it
- **owner** **pushes** the change to the repo

Pull requests



Forks

- making a remote copy of a remote repo
- contributing to the original by opening a pull request
- also: you can set a default on your github repo that everyone has to open a pull request and cannot commit to main directly
- also: needed to contribute to a repo that you don't have write access to

Before modifying local copy of your fork:

- sync your online fork with original online repo
- git pull

→ then make local modifications, add commit push to fork

→ open PR on original remote repo

Exercise 2 - Keep your partner!

- switch roles of **developer** and **owner**
- **developer** forks repo of **owner**
- **developer** clones their fork
- **developer** makes change and pushes it back to their fork
- **developer** opens a pull request in owner's repo
- **owner** reviews and merges pull request

Merge conflicts with the remote repository

- you can go on just like we did before:
 - inspect the merge conflict: open file
 - change file accordingly to solve the conflict: delete vs. modify and keep
 - add
 - commit
 - push it to remote

Advanced git commands

- **git revert:** brings back a previous version of your code as an additional commit
- **git reset:** undoing local changes and rewriting project history
- **git rebase:** solves the same problem as git merge but in a different way: gives you a much cleaner and a linear project history, **BUT** loses the context provided by a merge commit—you can't see when upstream changes were incorporated into the feature

